Library Management System: - 9/12/2024

Entities and Tables

Books Table:

Column Name	Data Type	Description
book_id (PK)	NUMBER	Unique identifier for each book.
title	VARCHAR2(255)	Title of the book.
author_id(FK)	Number	Author of the book.
genre	VARCHAR2(100)	Genre/category of the book.
Publisher	VARCHAR(255)	Publisher of the book.
publication_date	DATE	Date the book was published.
quantity_available	NUMBER(5)	Number of available copies.
quantity_borrowed	NUMBER(5)	Number of copies currently borrowed.

Authors Table;

Column Name	Data Type	Description
author_id (PK)	NUMBER	Unique identifier for each book.
author_name	VARCHAR2(255)	Name of Author.
genre	VARCHAR2(100)	Genre of Author.
nationality	VARCHAR2(100)	Nationality of Author.
date_of_birth	Date	Birth Date of author
website	VARCHAR2(255)	link to the author's official website, blog, or social media.

Members Table:

Column Name	Data Type	Description
member_id (PK)	NUMBER	Unique identifier for each member.
first_name	VARCHAR2(100)	First name of the member.
last_name	VARCHAR2(100)	Last name of the member.
email	VARCHAR2(255)	Email address of the member.
phone_number	VARCHAR2(15)	Contact phone number.
address	VARCHAR2(255)	Residential address.
membership_date	DATE	Date the member joined the library.
renewal_date	DATE	date by which the member must renew their membership.
membership_status	VARCHAR2(20)	Current status of membership.

Transactions Table:

Column Name	Data Type	Description
transaction_id (PK)	NUMBER	Unique identifier for each transaction.
book_id (FK)	NUMBER	Book being borrowed/returned.
member_id (FK)	NUMBER	Member who borrowed/returned the book.
staff_id(FK)	NUMBER	Staff who created the transaction.
borrow_date	DATE	Date the book was borrowed.
Book_due_date	DATE	Due date for returning the book.
Book_return_date	DATE	Date the book was returned (nullable).
status	VARCHAR2(15), CHECK ('borrowed','returned')	Status of the transaction.
late_charge	NUMBER(10,2)	Fine for returning the book late.
Late_charge_due	NUMBER(10,2)	Pendue late due amount
payment_mode	VARCHAR2(20)	Mode of payment

Staff Table:

Column Name	Data Type	Description
staff_id (PK)	NUMBER	Unique identifier for each staff.
first_name	VARCHAR2(100)	First name of the staff.
last_name	VARCHAR2(100)	Last name of the staff.
phone	VARCHAR2(15)	Phone no. of staff
email	VARCHAR2(100)	Email id of staff
address	VARCHAR2(255)	Adress of staff
salary	NUMBER (10, 2)	Salary of the staff.
department	VARCHAR2(100)	Department to which the staff belongs (e.g., HR, IT, etc.).
hire_date	DATE	Date when the staff was hired.

--TABLE CREATION--

```
create table Books( book_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
title VARCHAR2(255) NOT NULL, author VARCHAR2(255) NOT NULL, genre VARCHAR2(100),
published_year NUMBER(4), quantity_available NUMBER(5) DEFAULT 0 CHECK(quantity_available >= 0),
quantity_borrowed NUMBER(5) DEFAULT 0 CHECK (quantity_borrowed >= 0) );
```

```
CREATE TABLE Members( member_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
first_name VARCHAR2(100) NOT NULL, last_name VARCHAR2(100) NOT NULL, email VARCHAR2(100)
UNIQUE NOT NULL, phone_number VARCHAR2(15) NOT NULL, address VARCHAR2(255),
membership_date DATE DEFAULT SYSDATE NOT NULL, renewal_date DATE, membership_status
VARCHAR2(20) DEFAULT 'active' CHECK (membership_status IN('ACTIVE', 'EXPIRED', 'SUSPENDED')) );
```

```
CREATE TABLE Staffs( staff_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, first_name
VARCHAR2(100) NOT NULL, last_name VARCHAR2(100) NOT NULL, phone VARCHAR2(15), email
```

VARCHAR2(100), address VARCHAR2(255), hire_date DATE NOT NULL, salary NUMBER(10, 2) NOT NULL, department VARCHAR2(100) NOT NULL);

10/12/24

CREATE TABLE Transactions (transaction_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, book_id NUMBER NOT NULL REFERENCES Books(book_id), member_id NUMBER NOT NULL REFERENCES Members(member_id), staff_id NUMBER REFERENCES Staffs(staff_id), borrow_date DATE DEFAULT SYSDATE NOT NULL, due_date DATE NOT NULL, return_date DATE, status VARCHAR2(15) DEFAULT 'borrowed' CHECK (status IN ('borrowed', 'returned')));

CREATE TABLE Fines(fine_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, transaction_id NUMBER NOT NULL REFERENCES Transactions(transaction_id), member_id NUMBER NOT NULL REFERENCES Members(member_id), fine_amount NUMBER(10,2), fine_date DATE DEFAULT SYSDATE NOT NULL, is_paid CHAR DEFAULT 'N' CHECK (is_paid IN ('N','Y')), fine_paid_date DATE);

--SP--

--Bookborrow--

CREATE OR REPLACE PROCEDURE BorrowBook(p_book_id IN Books.book_id%Type, p_member_id IN Members.member_id%Type, p_staff_id IN Staffs.staff_id%Type, p_due_date IN Transactions.due_date%Type) IS v_available_quantity NUMBER; BEGIN SELECT quantity_available INTO v_available_quantity FROM Books WHERE book_id = p_book_id; if v_available_quantity < 0 THEN RAISE_APPLICATION_ERROR(-20001, 'Book is not available in stock'); END IF;

```
     INSERT INTO Transactions(book_id, member_id, staff_id, due_date)
      VALUES(p_book_id, p_member_id, p_staff_id, sysdate + 14);

      UPDATE Books
      SET quantity_available = quantity_available - 1,
      quantity_borrowed = quantity_borrowed + 1
      WHERE book_id = p_book_id;

      DBMS_OUTPUT.PUT_LINE('Book borrowed successfully');

      EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

  END;
  /
```

--BookReturn--

CREATE OR REPLACE PROCEDURE ReturnBook(p_transaction_id IN Transactions.transaction_id%type)
IS

v_book_id Transactions.book_id%Type; v_member_id Transactions.member_id%Type; v_due_date Transactions.due_date%Type; v_days_late NUMBER; v_fine_amount NUMBER; BEGIN SELECT book_id, member_id, due_date INTO v_book_id, v_member_id, v_due_date FROM Transactions WHERE transaction_id = p_transaction_id;

```
   IF v_due_date < sysdate THEN
    v_days_late := sysdate - v_due_date;
    v_fine_amount := 10 * v_days_late;

    INSERT INTO Fines(transaction_id, member_id, fine_amount)
    VALUES(p_transaction_id, v_member_id, v_fine_amount);

    DBMS_OUTPUT.PUT_LINE('Fine Imposed :' || v_fine_amount);
    ELSE
    DBMS_OUTPUT.PUT_LINE('No Fine Imposed');
    END IF;
UPDATE Transactions
SET status = 'returned', return_date = sysdate
WHERE transaction_id = p_transaction_id;

DBMS_OUTPUT.PUT_LINE('Book returned successfully');

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: Transaction does not exist or Invalid');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);


END; /
```

--fines--

CREATE OR REPLACE PROCEDURE PayFine(p_fine_id IN Fines.fine_id%Type)

IS

v_rows_updated NUMBER;

BEGIN

UPDATE Fines SET is_paid = 'Y', fine_paid_date = sysdate

```
WHERE fine_id = p_fine_id AND is_paid = 'N';

v_rows_updated := SQL%ROWCOUNT;
IF v_rows_updated > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Fine Paid Successfully');
ELSE
    DBMS_OUTPUT.PUT_LINE('Fine ID not found or already paid');
END IF;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);


END;
```

11/12/2024

--Triggers--

```
CREATE OR REPLACE TRIGGER UpdateBookAvailabilityAfterReturn

AFTER UPDATE OF status ON Transactions

FOR EACH ROW

BEGIN

        IF :NEW.status = 'returned' THEN

        UPDATE Books

        SET quantity_available = quantity_available + 1, quantity_borrowed = quantity_borrowed - 1

        WHERE book_id = :NEW.book_id;

        END IF;

 END;
```

--PACKAGE--

--Specification--

```
CREATE OR REPLACE PACKAGE LibraryManagement

IS
```

PROCEDURE BorrowBook (p_book_id IN Books.book_id%Type, p_member_id IN Members.member_id%Type, p_staff_id IN Staffs.staff_id%Type, p_due_date IN Transactions.due_date%Type);

PROCEDURE ReturnBook(p_transaction_id IN Transactions.transaction_id%type);

PROCEDURE PayFine(p_fine_id IN Fines.fine_id%Type);

END LibraryManagement;

/

--Body--

CREATE OR REPLACE PACKAGE BODY LibraryManagement

IS

--procedure for borrowing the book--

PROCEDURE BorrowBook(p_book_id IN Books.book_id%Type, p_member_id IN Members.member_id%Type, p_staff_id IN Staffs.staff_id%Type, p_due_date IN Transactions.due_date%Type)

IS

v_available_quantity NUMBER; BEGIN SELECT quantity_available INTO v_available_quantity FROM Books WHERE book_id = p_book_id; if v_available_quantity < 0 THEN RAISE_APPLICATION_ERROR(-20001, 'Book is not available in stock'); END IF;

```
INSERT INTO Transactions(book_id, member_id, staff_id, due_date)
              VALUES(p_book_id, p_member_id, p_staff_id, sysdate + 14);

UPDATE Books SET quantity_available = quantity_available - 1,

quantity_borrowed = quantity_borrowed + 1
              WHERE book_id = p_book_id;

              DBMS_OUTPUT.PUT_LINE('Book borrowed successfully');
```

Commit;

```
              EXCEPTION
                WHEN OTHERS THEN
                    DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);
```

Rollback;

```
          END BorrowBook;
```

--procedure for returning the book--

```
PROCEDURE ReturnBook(
    p_transaction_id IN Transactions.transaction_id%type)
    IS
        v_cnt NUMBER;
        v_book_id Transactions.book_id%Type;
        v_member_id Transactions.member_id%Type;
        v_due_date Transactions.due_date%Type;
        v_status Transactions.status%Type;
        v_days_late NUMBER;
        v_fine_amount NUMBER;
    BEGIN
        SELECT book_id, member_id, due_date, status INTO v_book_id, v_member_id,
v_due_date, v_status
        FROM Transactions
        WHERE transaction_id = p_transaction_id;

        IF v_status = 'returned' THEN
        RAISE_APPLICATION_ERROR(-20002, 'The book has already been returned.');
        END IF;

            IF v_due_date < sysdate THEN
            v_days_late := sysdate - v_due_date;
            v_fine_amount := 10 * v_days_late;

            INSERT INTO Fines(transaction_id, member_id, fine_amount)
            VALUES(p_transaction_id, v_member_id, v_fine_amount);

            DBMS_OUTPUT.PUT_LINE('Fine Imposed :' || v_fine_amount);
            ELSE
            DBMS_OUTPUT.PUT_LINE('No Fine Imposed');
            END IF;
        UPDATE Transactions
        SET status = 'returned', return_date = sysdate
        WHERE transaction_id = p_transaction_id;

        DBMS_OUTPUT.PUT_LINE('Book returned successfully');
```

Commit;

```
        EXCEPTION
            WHEN NO_DATA_FOUND THEN
```

```
Rollback;
                DBMS_OUTPUT.PUT_LINE('ERROR: Transaction does not exist or
Invalid');
            WHEN OTHERS THEN

Rollback;
                DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END ReturnBook;

--procedure to payfine--

PROCEDURE PayFine(p_fine_id IN Fines.fine_id%Type)
IS
    v_rows_updated NUMBER;
BEGIN
    UPDATE Fines
    SET is_paid = 'Y', fine_paid_date = sysdate
    WHERE fine_id = p_fine_id
    AND is_paid = 'N';

    v_rows_updated := SQL%ROWCOUNT;
    IF v_rows_updated > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Fine Paid Successfully');

Commit;
    ELSE
        DBMS_OUTPUT.PUT_LINE('Fine ID not found or already paid');
    END IF;

    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('ERROR: ' || SQLERRM);

Rollback;

END PayFine;


END LibraryManagement; /
```

--INSERTS--

--BOOKS--

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('The God of Small Things', 'Arundhati Roy', 'Fiction', 1997, 5, 5);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('Wings of Fire', 'A.P.J. Abdul Kalam', 'Autobiography', 1999, 8, 2);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('The White Tiger', 'Aravind Adiga', 'Fiction', 2008, 2, 3);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('India After Gandhi', 'Ramachandra Guha', 'History', 2007, 6, 1);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('The Alchemist', 'Paulo Coelho', 'Fiction', 1988, 10, 0);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('The Great Gatsby', 'F. Scott Fitzgerald', 'Fiction', 1925, 10, 3);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('1984', 'George Orwell', 'Dystopian', 1949, 5, 3);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('To Kill a Mockingbird', 'Harper Lee', 'Fiction', 1960, 8, 2);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('The Catcher in the Rye', 'J.D. Salinger', 'Fiction', 1951, 6, 4);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('Moby-Dick', 'Herman Melville', 'Adventure', 1851, 7, 1);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('War and Peace', 'Leo Tolstoy', 'Historical Fiction', 1869, 12, 5);

INSERT INTO Books(title, author, genre, published_year, quantity_available, quantity_borrowed) VALUES ('Pride and Prejudice', 'Jane Austen', 'Romance', 1813, 9, 1);

--Members--

INSERT INTO MEMBERS (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, ADDRESS, MEMBERSHIP_DATE, RENEWAL_DATE, MEMBERSHIP_STATUS) VALUES ('Rajesh','Kumar','rajesh.kumar@example.com','9876543210','Delhi',to_date('2022-01-15', 'YYYY-MM-DD'),to_date('2023-01-15', 'YYYY-MM-DD'),'EXPIRED');

INSERT INTO MEMBERS (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, ADDRESS, MEMBERSHIP_DATE, RENEWAL_DATE, MEMBERSHIP_STATUS) VALUES ('Anita','Sharma','anita.sharma@example.com','8765432109','Mumbai',to_date('2021-10-10', 'YYYY-MM-DD'),to_date('2022-10-10', 'YYYY-MM-DD'),'EXPIRED');

INSERT INTO MEMBERS (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, ADDRESS, MEMBERSHIP_DATE, RENEWAL_DATE, MEMBERSHIP_STATUS) VALUES ('Sunil','Verma','sunil.verma@example.com','7654321098','Kolkata',to_date('2023-04-20', 'YYYY-MM-DD'),to_date('2025-04-20', 'YYYY-MM-DD'),'ACTIVE');

INSERT INTO MEMBERS (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, ADDRESS, MEMBERSHIP_DATE, RENEWAL_DATE, MEMBERSHIP_STATUS) VALUES ('Meera','Nair','meera.nair@example.com','6543210987','Chennai',to_date('2020-08-01', 'YYYY-MM-DD'),null,'SUSPENDED');

INSERT INTO MEMBERS (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, ADDRESS, MEMBERSHIP_DATE, RENEWAL_DATE, MEMBERSHIP_STATUS) VALUES ('Priya','Singh','priya.singh@example.com','5432109876','Bangalore',to_date('2023-06-12', 'YYYY-MM-DD'),to_date('2025-06-12', 'YYYY-MM-DD'),'ACTIVE');

INSERT INTO Members(first_name, last_name, email, phone_number, address, membership_date, renewal_date, membership_status) VALUES ('Pankaj', 'Kishore', 'pankaj.kishore@example.com', '9875553210', 'Delhi', TO_DATE('2018-05-01', 'YYYY-MM-DD'), TO_DATE('2025-05-01', 'YYYY-MM-DD'), 'ACTIVE');

--staffs--

INSERT INTO Staffs(first_name, last_name, phone, email, address, hire_date, salary, department) VALUES ('Amit', 'Gupta', '9998887771', 'amit.gupta@library.com', 'Delhi, India', TO_DATE('2015-05-10', 'YYYY-MM-DD'), 45000, 'Administration');

INSERT INTO Staffs(first_name, last_name, phone, email, address, hire_date, salary, department) VALUES ('Nisha', 'Kapoor', '8887776662', 'nisha.kapoor@library.com', 'Mumbai, India', TO_DATE('2018-03-22', 'YYYY-MM-DD'), 40000, 'Library Assistance');

INSERT INTO Staffs(first_name, last_name, phone, email, address, hire_date, salary, department) VALUES ('Ravi', 'Chauhan', '7776665553', 'ravi.chauhan@library.com', 'Lucknow, India', TO_DATE('2020-11-15', 'YYYY-MM-DD'), 35000, 'Security');

INSERT INTO Staffs(first_name, last_name, phone, email, address, hire_date, salary, department) VALUES ('Sneha', 'Reddy', '6665554444', 'sneha.reddy@library.com', 'Hyderabad, India', TO_DATE('2019-06-01', 'YYYY-MM-DD'), 50000, 'Technology Support');

--Transactions--

```
INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (1, 1, 1, TO_DATE('2023-11-15', 'YYYY-MM-DD'), TO_DATE('2023-11-25', 'YYYY-MM-DD'), NULL,
'borrowed'); --not yet

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (2, 2, 2, TO_DATE('2023-10-01', 'YYYY-MM-DD'), TO_DATE('2023-10-10', 'YYYY-MM-DD'),
TO_DATE('2023-10-12', 'YYYY-MM-DD'), 'returned'); --late

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (3, 3, 3, TO_DATE('2023-12-01', 'YYYY-MM-DD'), TO_DATE('2023-12-10', 'YYYY-MM-DD'), NULL,
'borrowed'); --not yet

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (4, 4, 4, TO_DATE('2023-10-01', 'YYYY-MM-DD'), TO_DATE('2023-10-10', 'YYYY-MM-DD'),
TO_DATE('2023-10-08', 'YYYY-MM-DD'), 'returned'); --early

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (5, 5, 1, TO_DATE('2023-11-28', 'YYYY-MM-DD'), TO_DATE('2023-12-08', 'YYYY-MM-DD'), NULL,
'borrowed'); --not yet

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (6, 1, 2, TO_DATE('2023-09-05', 'YYYY-MM-DD'), TO_DATE('2023-09-15', 'YYYY-MM-DD'),
TO_DATE('2023-09-17', 'YYYY-MM-DD'), 'returned'); --late

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (7, 2, 3, TO_DATE('2023-08-10', 'YYYY-MM-DD'), TO_DATE('2023-08-20', 'YYYY-MM-DD'),
TO_DATE('2023-08-21', 'YYYY-MM-DD'), 'returned');--late

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (8, 3, 4, TO_DATE('2023-07-01', 'YYYY-MM-DD'), TO_DATE('2023-07-10', 'YYYY-MM-DD'),
TO_DATE('2023-07-12', 'YYYY-MM-DD'), 'returned'); --late

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (9, 4, 1, TO_DATE('2023-11-05', 'YYYY-MM-DD'), TO_DATE('2023-11-15', 'YYYY-MM-DD'),
TO_DATE('2023-11-17', 'YYYY-MM-DD'), 'returned'); --late

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (10, 5, 2, TO_DATE('2023-06-20', 'YYYY-MM-DD'), TO_DATE('2023-06-30', 'YYYY-MM-DD'),
TO_DATE('2023-07-02', 'YYYY-MM-DD'), 'returned'); --late

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (4, 4, 1, TO_DATE('2022-12-12', 'YYYY-MM-DD'), TO_DATE('2022-12-26', 'YYYY-MM-DD'),
TO_DATE('2022-12-24', 'YYYY-MM-DD'), 'returned'); --early

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status)
VALUES (4, 4, 2, TO_DATE('2023-02-01', 'YYYY-MM-DD'), TO_DATE('2023-02-15', 'YYYY-MM-DD'),
TO_DATE('2023-02-012', 'YYYY-MM-DD'), 'returned'); --early
```

INSERT INTO Transactions(book_id, member_id, staff_id, borrow_date, due_date, return_date, status) VALUES (4, 4, 4, TO_DATE('2023-10-05', 'YYYY-MM-DD'), TO_DATE('2023-10-19', 'YYYY-MM-DD'), TO_DATE('2023-10-08', 'YYYY-MM-DD'), 'returned'); --early

12/12/24

--Fines--

INSERT INTO Fines(transaction_id, member_id, fine_amount, fine_date, is_paid, fine_paid_date) VALUES (2, 2, 20.00, TO_DATE('2023-10-13', 'YYYY-MM-DD'), 'Y', TO_DATE('2023-10-14', 'YYYY-MM-DD'));

INSERT INTO Fines(transaction_id, member_id, fine_amount, fine_date, is_paid, fine_paid_date) VALUES (11, 1, 20.00, TO_DATE('2023-09-17', 'YYYY-MM-DD'), 'Y', TO_DATE('2023-09-18', 'YYYY-MM-DD'));

INSERT INTO Fines(transaction_id, member_id, fine_amount, fine_date, is_paid, fine_paid_date) VALUES (12, 2, 10.00, TO_DATE('2023-08-21', 'YYYY-MM-DD'), 'Y', TO_DATE('2023-08-22', 'YYYY-MM-DD'));

INSERT INTO Fines(transaction_id, member_id, fine_amount, fine_date, is_paid, fine_paid_date) VALUES (13, 3, 20.00, TO_DATE('2023-07-12', 'YYYY-MM-DD'), 'Y', TO_DATE('2023-07-13', 'YYYY-MM-DD'));

INSERT INTO Fines(transaction_id, member_id, fine_amount, fine_date, is_paid, fine_paid_date) VALUES (14, 4, 20.00, TO_DATE('2023-11-17', 'YYYY-MM-DD'), 'Y', TO_DATE('2023-11-18', 'YYYY-MM-DD'));

INSERT INTO Fines(transaction_id, member_id, fine_amount, fine_date, is_paid, fine_paid_date) VALUES (15, 5, 20.00, TO_DATE('2023-07-02', 'YYYY-MM-DD'), 'Y', TO_DATE('2023-07-03', 'YYYY-MM-DD'));

Commits needs to be checked in SPS